CLASE 11

Visión Artificial con OpenCV

Guía de Autoaprendizaje — scripts de la clase

Robótica Industrial · FPTEN27 · Universidad Metropolitana

Esta guía acompaña los siete scripts que se ejecutaron en la Clase 11. La idea es que los corras en tu computador, cambies sus parámetros y compruebes por ti mismo cada concepto de la clase: la imagen como matriz de números, el espacio HSV, los filtros, la detección de bordes, la segmentación y la medición de distancia con una cámara.

Cada sección sigue el mismo orden: **qué hace** el script, **las líneas clave** (solo las funciones esenciales, no hace falta memorizar la sintaxis), **Juega con el código** (experimentos cortos marcados E1, E2…) y **Para pensar**. Al final hay ejercicios de repaso con sus respuestas.

1. Preparación del entorno

> ⚠ **Requisitos**
> Python 3 instalado (verifícalo con `python --version`), una cámara web y conexión a internet para descargar OpenCV.

1 **Instalar OpenCV y NumPy**

Abre la terminal de Windows (CMD o PowerShell) y ejecuta:

```
pip install opencv-python numpy
```

2 **Verificar la instalación**

```
python -c "import cv2; print(cv2.__version__)"
```

3 **Ejecutar un script**

Guarda los siete scripts en una misma carpeta, abre la terminal en esa carpeta y ejecuta, por ejemplo:

```
python filtros.py
```

ℹ Cómo trabajar con los scripts

• Para cerrar un script, haz clic sobre su ventana y presiona **Q**. Si presionas Q en la terminal no pasa nada: OpenCV solo escucha el teclado en sus propias ventanas.

• Antes de modificar un script, guarda una copia (por ejemplo `filtros_original.py`) para poder volver al punto de partida.

• Todos tienen la misma estructura: abrir la cámara con `cv2.VideoCapture(0)`, leer un cuadro (*frame*) dentro de un `while True`, procesarlo, mostrarlo con `cv2.imshow()` y repetir.

1.1 Mapa de los scripts

Script	Tema de la clase	Qué verás
`filtros.py`	Filtros y convolución	Original, grises, blur y bordes en una sola ventana
`histograma.py`	La imagen digital / HSV	Cómo se reparten las intensidades de cada canal
`deteccion_bordes.py`	Detección de bordes	El detector Canny en tiempo real
`deteccion_color.py`	Espacio HSV y segmentación	Máscara de un color, contorno, área y centroide
`tracking.py`	Segmentación	Seguimiento de un objeto con su rastro
`distancia.py`	Del código al mundo real	Distancia a una hoja usando una sola cámara
`distancia_rostros.py`	Detección de patrones	Rostros con Haar Cascade y su distancia

2. La imagen digital: una cámara ve números

Para OpenCV, cada cuadro que entrega la cámara es una **matriz de NumPy**. Una imagen a color de 640 × 480 tiene 480 filas, 640 columnas y 3 canales; cada valor es un entero sin signo de 8 bits (`uint8`), de 0 a 255 [11].

- 640 × 480 = 307 200 píxeles; × 3 canales = **921 600 bytes por cuadro**.
- A 30 cuadros por segundo son unos 27,6 MB/s. Por eso el reto es *reducir y destilar*: quedarse solo con la información útil.
- Un píxel se lee como `frame[fila, columna]`, es decir `frame[y, x]`. El origen (0, 0) está en la **esquina superior izquierda** y el eje y crece **hacia abajo** [11].

★ OpenCV guarda los colores en orden BGR

Al leer un píxel a color, OpenCV devuelve **[Azul, Verde, Rojo]**, no [R, G, B] [11]. Es la causa más común de colores "cambiados" cuando se mezcla OpenCV con otras librerías.

2.1 Script de exploración

Este script corto no se vio en clase, pero sirve para mirar los números de un cuadro. Cópialo en un archivo `explorar_pixel.py`:

```
import cv2

cap = cv2.VideoCapture(0)
ret, frame = cap.read()        # un solo cuadro
cap.release()

print("Tamano:", frame.shape)  # (filas, columnas, canales)
print("Tipo:", frame.dtype)
print("Pixel central [B, G, R]:", frame[240, 320])
```

E1 **Lee el píxel central**
Pon un objeto rojo justo frente a la cámara y ejecuta el script. ¿Cuál de los tres números es el mayor? ¿Coincide con el orden BGR?

E2 **Tapa la cámara**
Ejecútalo con la cámara tapada y luego apuntando a una lámpara. ¿Qué valores obtienes en cada caso?

E3 **Cambia la posición**
Cambia `frame[240, 320]` por `frame[0, 0]` (esquina superior izquierda). ¿Qué pasa si escribes `frame[600, 100]` en una imagen de 480 filas?

3. filtros.py — el preprocesamiento paso a paso

Muestra en una cuadrícula de 2 × 2 las cuatro etapas del preprocesamiento: la imagen original, la escala de grises, el suavizado gaussiano y los bordes de Canny.

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)
print("Presiona Q para salir")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Los cuatro procesamientos
    gris      = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    blur      = cv2.GaussianBlur(gris, (15, 15), 0)
    bordes    = cv2.Canny(blur, 50, 150)

    # Convertir grises a BGR para poder apilarlos con la imagen color
    gris_bgr   = cv2.cvtColor(gris,   cv2.COLOR_GRAY2BGR)
    blur_bgr   = cv2.cvtColor(blur,   cv2.COLOR_GRAY2BGR)
    bordes_bgr = cv2.cvtColor(bordes, cv2.COLOR_GRAY2BGR)

    # Etiquetas en cada imagen
    def etiquetar(img, texto):
        cv2.putText(img, texto, (10, 30),
```

```
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2)
        return img

    etiquetar(frame,      "1. Original")
    etiquetar(gris_bgr,   "2. Escala de grises")
    etiquetar(blur_bgr,   "3. Blur Gaussiano")
    etiquetar(bordes_bgr, "4. Bordes Canny")

    # Apilar en cuadricula 2x2
    fila1 = np.hstack([frame,      gris_bgr])
    fila2 = np.hstack([blur_bgr,   bordes_bgr])
    grid  = np.vstack([fila1, fila2])

    # Reducir tamano para que quepa en pantalla
    alto, ancho = grid.shape[:2]
    grid = cv2.resize(grid, (ancho // 2, alto // 2))

    cv2.imshow("Pipeline de Vision Artificial", grid)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Líneas clave

- **`cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)`**: pasa de 3 canales a 1. Menos datos y, para buscar bordes, el color no hace falta.
- **`cv2.GaussianBlur(gris, (15, 15), 0)`**: suaviza la imagen deslizando una matriz pequeña (el *kernel*) de 15 × 15 sobre cada píxel; es la **convolución** de la lámina "Filtros y convolución" [3]. El tamaño del kernel debe ser **positivo e impar** [3].
- **`cv2.Canny(blur, 50, 150)`**: detecta bordes (sección 5).
- **`np.hstack`** y **`np.vstack`** solo pegan las cuatro imágenes para verlas juntas; no procesan nada.

Juega con el código

E1 — **Tamaño del kernel**
Cambia **`(15, 15)`** por **`(3, 3)`** y luego por **`(31, 31)`**. Observa el cuadro 3 (blur) y el cuadro 4 (bordes): ¿qué pasa con la cantidad de bordes al suavizar más?

E2 — **Kernel par**
Prueba **`(10, 10)`**. El programa se detiene con un error: lee el mensaje y ubica la condición que no se cumple (**`ksize.width % 2 == 1`**).

E3 — **Movimiento**
Mueve la mano rápido frente a la cámara. ¿En cuál de los cuatro cuadros desaparece primero la forma de tu mano?

> **Para pensar**
> ¿Por qué se suaviza la imagen **antes** de buscar bordes? Piensa en el ruido de "sal y pimienta" de la clase: cada píxel erróneo es un cambio brusco de intensidad.

4. histograma.py — cómo se reparte la luz

Un histograma cuenta cuántos píxeles tienen cada nivel de intensidad, de 0 a 255 [10]. Este script dibuja tres curvas, una por canal: el eje horizontal es la intensidad (oscuro a la izquierda, claro a la derecha) y el eje vertical, cuántos píxeles tienen esa intensidad.

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)
print("Presiona Q para salir")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    hist_img = np.zeros((300, 512, 3), dtype=np.uint8)

    colores = [(0, (255, 0, 0)), (1, (0, 255, 0)), (2, (0, 0, 255))]

    for canal, color in colores:
        hist = cv2.calcHist([frame], [canal], None, [256], [0, 256])
        hist = cv2.normalize(hist, hist, 0, 300, cv2.NORM_MINMAX)
        hist = hist.flatten()   # convierte (256,1) a (256,) -- fix del error

        for x in range(1, 256):
            cv2.line(
                hist_img,
                (x * 2 - 2, 300 - int(hist[x - 1])),
                (x * 2,     300 - int(hist[x])),
                color, 1
            )

    cv2.imshow("Camara", frame)
    cv2.imshow("Histograma R-G-B", hist_img)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Líneas clave

- **`cv2.calcHist([frame], [canal], None, [256], [0, 256])`**: histograma de un canal (0 = B, 1 = G, 2 = R), con 256 barras que cubren de 0 a 255 [10].
- **`cv2.normalize(..., 0, 300, ...)`**: escala la curva para que quepa en los 300 píxeles de alto de la ventana. Solo es para dibujar.
- El bucle **`for x in range(1, 256)`** une con líneas los 256 valores de cada curva.

Juega con el código

E1 **Hoja blanca**
Pon una hoja blanca frente a la cámara. ¿Hacia dónde se mueven las tres curvas?

E2 **Oscuridad**
Tapa la cámara a medias. ¿Qué pasa con las curvas?

E3 **Objeto rojo**
Muestra un objeto rojo grande. ¿Qué curva se separa de las otras dos?

E4 **Mismo objeto, otra luz**
Con el mismo objeto rojo, enciende y apaga una lámpara (o acércate y aléjate de la ventana). ¿Cambian solo las curvas del rojo o las tres a la vez?

> ★ **Conexión con la lámina "Espacio HSV"**
> Lo que viste en E4 es justo el problema de RGB: el mismo objeto cambia sus tres valores cuando cambia la luz. HSV separa el **tono** (H) del **brillo** (V), y por eso se usa para detectar colores (sección 6).

5. deteccion_bordes.py — el detector Canny

Abre dos ventanas: una que alterna entre la imagen normal y los bordes (tecla **B**) y otra que muestra siempre los bordes.

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)
print("Presiona Q para salir")
print("Presiona B para alternar entre vista normal y bordes")

modo_bordes = False

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Convertir a escala de grises
    gris = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Suavizar para reducir ruido antes de detectar bordes
    suavizado = cv2.GaussianBlur(gris, (5, 5), 0)

    # Detector de bordes Canny
    bordes = cv2.Canny(suavizado, 50, 150)

    # Mostrar texto indicando el modo
```

```
    texto = "MODO: BORDES (B para cambiar)" if modo_bordes else "MODO: NORMAL (B para
cambiar)"
    cv2.putText(frame, texto, (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 2)

    if modo_bordes:
        cv2.imshow("Deteccion de Bordes", bordes)
    else:
        cv2.imshow("Deteccion de Bordes", frame)

    cv2.imshow("Bordes Canny", bordes)

    key = cv2.waitKey(1) & 0xFF
    if key == ord('q'):
        break
    elif key == ord('b'):
        modo_bordes = not modo_bordes

cap.release()
cv2.destroyAllWindows()
```

Las cuatro etapas de Canny

Una sola línea, `cv2.Canny(suavizado, 50, 150)`, ejecuta cuatro etapas [2]:

1. **Reducción de ruido** con un filtro gaussiano.
2. **Gradiente de intensidad**: con filtros de Sobel calcula cuánto y en qué dirección cambia el brillo en cada píxel. Un borde es donde ese cambio es máximo (la curva de la lámina "Detección de bordes").
3. **Supresión de no máximos**: deja los bordes de un píxel de grosor.
4. **Umbral con histéresis**, con los dos números de la función:

Gradiente del píxel	Resultado
Mayor que 150 (umbral alto)	Borde seguro
Menor que 50 (umbral bajo)	Se descarta
Entre 50 y 150	Es borde solo si está conectado a un borde seguro

Juega con el código

E1 **Umbrales**
Prueba las combinaciones de la tabla y anota lo que observas.

Umbrales en Canny	¿Muchos o pocos bordes?	¿Qué bordes desaparecen?
(10, 30)		
(50, 150)		
(100, 200)		
(200, 250)		

E2 **Sin suavizado**

Cambia `cv2.Canny(suavizado, 50, 150)` por `cv2.Canny(gris, 50, 150)`. ¿Qué aparece en zonas lisas como la pared o la mesa?

E3 **Textura**

Apunta la cámara a una tela con textura y luego a una hoja lisa. ¿Qué pasa con la cantidad de bordes?

ℹ Para pensar

Si subes ambos umbrales, solo sobreviven los cambios de brillo más fuertes. ¿Por qué eso puede ser útil en una línea de producción con fondo uniforme y por qué es un problema con piezas metálicas que reflejan la luz?

6. deteccion_color.py — segmentación por color en HSV

Busca un objeto **naranja**, crea una máscara en blanco y negro, encuentra su contorno y calcula su centroide. Es el script que conecta la imagen con una posición que un robot podría usar.

```
import cv2
import numpy as np

cap = cv2.VideoCapture(0)
print("Presiona Q para salir")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    hsv     = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    bajo    = np.array([5,  120, 100])
    alto    = np.array([20, 255, 255])
    mascara = cv2.inRange(hsv, bajo, alto)

    # Encontrar contornos del objeto detectado
    contornos, _ = cv2.findContours(
        mascara, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )

    if contornos:
        # Tomar el contorno mas grande
        c = max(contornos, key=cv2.contourArea)
        area = cv2.contourArea(c)

        # Solo procesar si el objeto es suficientemente grande
        if area > 500:
            # Calcular centroide
            M  = cv2.moments(c)
            cx = int(M["m10"] / M["m00"])
            cy = int(M["m01"] / M["m00"])

            # Dibujar contorno y centroide
```

```
            cv2.drawContours(frame, [c], -1, (0, 255, 0), 2)
            cv2.circle(frame, (cx, cy), 8, (0, 0, 255), -1)

            # Mostrar coordenadas en pantalla
            cv2.putText(
                frame,
                f"Centro: ({cx}, {cy})",
                (cx + 10, cy - 10),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.6, (0, 0, 255), 2
            )

            # Mostrar area
            cv2.putText(
                frame,
                f"Area: {int(area)} px",
                (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.7, (255, 255, 255), 2
            )

    cv2.imshow("Deteccion de Color", frame)
    cv2.imshow("Mascara", mascara)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

6.1 El espacio HSV en OpenCV

En OpenCV, **H va de 0 a 179**, mientras que **S y V van de 0 a 255** [1]. H es el tono (el color "puro"), S la saturación (0 = gris, 255 = color intenso) y V el brillo.

El rango del script, `[5, 120, 100]` a `[20, 255, 255]`, se lee así: tono entre 5 y 20 (naranja), saturación de al menos 120 y brillo de al menos 100.

¿Cómo encontrar el rango de otro color?

La documentación de OpenCV sugiere convertir a HSV el color que buscas y usar un rango de ±10 en H, con S y V desde 100 [1]:

```
import cv2, numpy as np
verde = np.uint8([[[0, 255, 0]]])                  # en orden B, G, R
print(cv2.cvtColor(verde, cv2.COLOR_BGR2HSV))      # [[[ 60 255 255]]]
# rango sugerido: [50, 100, 100] a [70, 255, 255]
```

Color puro (B, G, R)	H en OpenCV	Rango sugerido de H
Rojo (0, 0, 255)	0	0–10 y 170–179
Naranja (0, 128, 255)	15	5–25
Amarillo (0, 255, 255)	30	20–40
Verde (0, 255, 0)	60	50–70
Azul (255, 0, 0)	120	110–130

Valores de H calculados con `cv2.cvtColor` sobre colores puros. Los objetos reales no son colores puros: ajusta el rango mirando la ventana "Mascara".

6.2 Líneas clave

- `cv2.inRange(hsv, bajo, alto)`: crea la **máscara**: blanco (255) donde el píxel está dentro del rango y negro (0) donde no.
- `cv2.findContours(mascara, cv2.RETR_EXTERNAL, ...)`: agrupa los píxeles blancos conectados en contornos; `RETR_EXTERNAL` pide solo los contornos exteriores.
- `max(contornos, key=cv2.contourArea)`: se queda con el contorno de mayor área. El filtro `area > 500` ignora manchas pequeñas.
- `cv2.moments(c)`: el centroide es **cx = m10 / m00** y **cy = m01 / m00**, donde m00 es el área [5]. Son coordenadas en **píxeles**.

Juega con el código

E1 **Otro color**
Cambia el rango para detectar un objeto azul: `bajo = np.array([110, 100, 100])`, `alto = np.array([130, 255, 255])`.

E2 **Saturación mínima**
Baja la saturación mínima de 120 a 40. Mira la ventana "Mascara": ¿qué otras cosas del entorno (piel, madera) se cuelan?

E3 **Área mínima**
Cambia `area > 500` por `area > 5000` y aleja el objeto. ¿A qué distancia deja de detectarse?

E4 **Origen de coordenadas**
Lleva el objeto a cada esquina de la imagen y anota (cx, cy). ¿Dónde está el (0, 0)? ¿Cuánto vale cy abajo a la derecha?

> ⚠ **Reto opcional: el rojo da la vuelta**
> El rojo queda en los dos extremos de H (cerca de 0 y cerca de 179), así que hacen falta dos máscaras unidas:

```
m1 = cv2.inRange(hsv, np.array([0, 120, 100]),   np.array([10, 255, 255]))
m2 = cv2.inRange(hsv, np.array([170, 120, 100]), np.array([179, 255, 255]))
mascara = cv2.bitwise_or(m1, m2)
```

7. tracking.py — seguimiento con rastro

Sigue un objeto naranja cuadro a cuadro y dibuja el camino que recorrió. Parte de **`deteccion_color.py`** y agrega tres cosas: limpieza de la máscara, un círculo envolvente y la memoria de las últimas posiciones.

```
import cv2
import numpy as np
from collections import deque

cap = cv2.VideoCapture(0)
print("Presiona Q para salir")
print("Mueve un objeto naranja frente a la camara")

# Guardar los ultimos 64 centroides para dibujar el rastro
rastro = deque(maxlen=64)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    hsv     = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    bajo    = np.array([5,  120, 100])
    alto    = np.array([20, 255, 255])
    mascara = cv2.inRange(hsv, bajo, alto)

    # Eliminar ruido
    mascara = cv2.erode(mascara,  None, iterations=2)
    mascara = cv2.dilate(mascara, None, iterations=2)

    contornos, _ = cv2.findContours(
        mascara, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )

    centro = None

    if contornos:
        c    = max(contornos, key=cv2.contourArea)
        area = cv2.contourArea(c)

        if area > 500:
            ((x, y), radio) = cv2.minEnclosingCircle(c)
            M  = cv2.moments(c)
            cx = int(M["m10"] / M["m00"])
            cy = int(M["m01"] / M["m00"])
            centro = (cx, cy)

            # Dibujar circulo alrededor del objeto
            cv2.circle(frame, (int(x), int(y)), int(radio), (0, 255, 255), 2)
            # Dibujar centroide
            cv2.circle(frame, centro, 6, (0, 0, 255), -1)
            # Mostrar coordenadas
            cv2.putText(frame, f"({cx}, {cy})",
                        (cx + 10, cy - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)

    # Agregar centroide actual al rastro
    rastro.appendleft(centro)

    # Dibujar el rastro
```

```
    for i in range(1, len(rastro)):
        if rastro[i - 1] is None or rastro[i] is None:
            continue
        # Grosor decrece con la antiguedad del punto
        grosor = int(np.sqrt(64 / float(i + 1)) * 2.5)
        cv2.line(frame, rastro[i - 1], rastro[i], (0, 140, 255), grosor)

    cv2.imshow("Tracking", frame)
    cv2.imshow("Mascara",  mascara)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Líneas clave

- **`cv2.erode`** y luego **`cv2.dilate`**: la **erosión** borra los puntos blancos pequeños (ruido), pero también encoge el objeto; la **dilatación** le devuelve su tamaño. Como el ruido ya desapareció, no vuelve [4].
- **`cv2.minEnclosingCircle(c)`**: el círculo más pequeño que contiene el contorno [5].
- **`deque(maxlen=64)`**: una lista que guarda como máximo 64 elementos; al llegar uno nuevo con **`appendleft`**, el más viejo se descarta solo.
- Si en un cuadro no hay objeto, se guarda **`None`** y ese tramo del rastro no se dibuja. El grosor de la línea disminuye con la antigüedad del punto.

Juega con el código

E1 **Sin limpieza**
Pon **`iterations=0`** en las dos líneas. Compara la ventana "Mascara" con la original.

E2 **Limpieza exagerada**
Pon **`iterations=6`**. ¿Qué le pasa a un objeto naranja pequeño?

E3 **Memoria del rastro**
Cambia **`maxlen=64`** por **`16`** y luego por **`200`**. ¿Qué cambia al mover el objeto?

> **ℹ Para pensar**
> **`rastro[0]`** es la posición actual y **`rastro[1]`** la del cuadro anterior. ¿Cómo calcularías la velocidad del objeto en píxeles por cuadro? ¿Y en píxeles por segundo, si la cámara entrega 30 cuadros por segundo?

8. distancia.py — medir distancia con una sola cámara

Estima a qué distancia está una hoja de papel usando el **modelo de cámara estenopeica** (*pinhole*): un objeto se ve más pequeño en la imagen mientras más lejos está [9]. Por triángulos semejantes, el ancho en píxeles P de un objeto de ancho real W a una distancia D cumple [8]:

$$P = F \cdot W / D$$

F es la distancia focal de la cámara expresada en píxeles [7]. Como no la conocemos, se **calibra**: se pone la hoja a una distancia conocida (30 cm), se mide su ancho en píxeles y se despeja F. Desde ese momento, cada cuadro permite despejar D [8]:

Momento	Fórmula	En el script
Calibración (tecla C)	F = P · D / W	`focal = (w * 30.0) / ANCHO_REAL_CM`
Medición (cada cuadro)	D = W · F / P	`distancia = (ANCHO_REAL_CM * focal) / w`

> ★ **Ejemplo numérico**
> Al calibrar a 30 cm la hoja (W = 21 cm) mide P = 210 px, así que F = 210 · 30 / 21 = **300 px**. Si después la hoja mide 105 px, D = 21 · 300 / 105 = **60 cm**. La distancia es inversamente proporcional al ancho en píxeles: la mitad de ancho, el doble de distancia.

```
import cv2
import numpy as np

ANCHO_REAL_CM = 21.0
focal = None
distancia_calibracion = 30.0

def detectar_hoja(frame):
    gris      = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    suavizado = cv2.GaussianBlur(gris, (9, 9), 0)
    bordes    = cv2.Canny(suavizado, 20, 80)

    # Dilatar bordes para cerrar huecos
    kernel = np.ones((3, 3), np.uint8)
    bordes = cv2.dilate(bordes, kernel, iterations=1)

    contornos, _ = cv2.findContours(
        bordes, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
    )

    if not contornos:
        return None, None, bordes

    c    = max(contornos, key=cv2.contourArea)
    area = cv2.contourArea(c)

    # Mostrar area del contorno mas grande para diagnostico
    print(f"Area mayor contorno: {int(area)}", end="\r")

    if area < 3000:
        return None, None, bordes

    x, y, w, h = cv2.boundingRect(c)
```

```
    return (x, y, w, h), c, bordes

cap = cv2.VideoCapture(0)
print("Presiona Q para salir")
print("Presiona C para calibrar cuando la hoja este a 30 cm")
print("Presiona D para ver el debug de bordes")

modo_debug = False

while True:
    ret, frame = cap.read()
    if not ret:
        break

    bbox, contorno, bordes = detectar_hoja(frame)

    if bbox is not None:
        x, y, w, h = bbox
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

        if focal is not None:
            distancia = (ANCHO_REAL_CM * focal) / w
            cv2.putText(frame, f"Distancia: {distancia:.1f} cm",
                        (x, y - 40),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 255), 2)

        cv2.putText(frame, f"Ancho: {w} px  Area: {int(cv2.contourArea(contorno))} px",
                    (x, y - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 0), 2)

        estado = "CALIBRADO" if focal else "Presiona C para calibrar"
        color  = (0, 255, 0) if focal else (0, 0, 255)
        cv2.putText(frame, estado, (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, color, 2)
    else:
        cv2.putText(frame, "Sin hoja detectada", (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)

    if modo_debug:
        bordes_bgr = cv2.cvtColor(bordes, cv2.COLOR_GRAY2BGR)
        cv2.imshow("Debug bordes", bordes_bgr)

    cv2.imshow("Medicion de Distancia", frame)

    key = cv2.waitKey(1) & 0xFF
    if key == ord('q'):
        break
    elif key == ord('d'):
        modo_debug = not modo_debug
        if not modo_debug:
            cv2.destroyWindow("Debug bordes")
    elif key == ord('c'):
        if bbox is not None:
            x, y, w, h = bbox
            focal = (w * distancia_calibracion) / ANCHO_REAL_CM
            print(f"\nCalibrado OK. Focal: {focal:.1f} px  Ancho hoja: {w} px")
        else:
            print("\nNo se detecto hoja al presionar C")

cap.release()
cv2.destroyAllWindows()
```

Líneas clave

- `detectar_hoja()` usa el mismo pipeline de las secciones anteriores: grises, blur, Canny, contornos.
- `cv2.dilate` engrosa los bordes para **cerrar los huecos** y que el contorno de la hoja quede cerrado.
- `cv2.boundingRect(c)` devuelve el rectángulo recto que envuelve el contorno: (x, y) de la esquina superior izquierda, ancho w y alto h [5]. El ancho **w** es la P de la fórmula.
- Teclas: **C** calibra (con la hoja a 30 cm), **D** muestra los bordes para diagnosticar, **Q** sale.

Juega con el código

Coloca la hoja sobre un fondo oscuro y uniforme, calíbrala a 30 cm medidos con regla y completa la tabla:

Distancia real (cm)	Distancia estimada (cm)	Error (%)
30 (calibración)		
40		
50		
70		
90		

E1 **Hoja inclinada**
Inclina la hoja hacia un lado. ¿La distancia estimada sube o baja? ¿Por qué?

E2 **Tamaño de la hoja**
El script supone una hoja A4 (21,0 cm de ancho). Una hoja carta mide 21,6 cm. Si usas carta sin cambiar `ANCHO_REAL_CM`, ¿qué error introduces?

E3 **Otro objeto**
Cambia `ANCHO_REAL_CM` por el ancho de un libro o cuaderno, recalibra y mide.

> **i Para pensar**
> La fórmula solo funciona si el objeto está de frente a la cámara y si conocemos su ancho real. ¿Qué supuesto falla con un robot que ve piezas de distintos tamaños sobre una banda transportadora?

9. distancia_rostros.py — detección de patrones con Haar Cascade

Aplica la misma fórmula de distancia, pero el objeto ahora es un **rostro**, detectado con un **clasificador Haar Cascade** preentrenado que trae OpenCV. Este método, propuesto por Viola y Jones (2001), revisa la imagen en ventanas y evalúa cada una en etapas: si una ventana falla la primera etapa se descarta de inmediato, y solo las que pasan todas se consideran rostro [6].

```
import cv2

ANCHO_CARA_CM = 14.0
focal = None
distancia_calibracion = 30.0

detector = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
)

cap = cv2.VideoCapture(0)
print("Presiona Q para salir")
print("Presiona C para calibrar cuando tu cara este a 30 cm de la camara")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    gris    = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    rostros = detector.detectMultiScale(
        gris, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30)
    )

    for (x, y, w, h) in rostros:
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)

        if focal is not None:
            distancia = (ANCHO_CARA_CM * focal) / w
            color_dist = (0, 255, 0) if distancia < 50 else (0, 165, 255)
            cv2.putText(frame, f"Distancia: {distancia:.1f} cm",
                        (x, y - 35),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.75, color_dist, 2)

        cv2.putText(frame, f"Ancho: {w} px",
                    (x, y - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

        # Centroide
        cx = x + w // 2
        cy = y + h // 2
        cv2.circle(frame, (cx, cy), 5, (0, 0, 255), -1)

    # Estado
    if focal is None:
        cv2.putText(frame, "SIN CALIBRAR — Presiona C a 30 cm",
                    (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 0, 255), 2)
    else:
        cv2.putText(frame, "CALIBRADO",
                    (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 255, 0), 2)

    cv2.putText(frame, f"Rostros: {len(rostros)}",
                (10, 60),
                cv2.FONT_HERSHEY_SIMPLEX, 0.65, (255, 255, 0), 2)

    cv2.imshow("Distancia por Rostro", frame)

    key = cv2.waitKey(1) & 0xFF
    if key == ord('q'):
        break
```

```
    elif key == ord('c'):
        if len(rostros) > 0:
            w = rostros[0][2]
            focal = (w * distancia_calibracion) / ANCHO_CARA_CM
            print(f"Calibrado OK. Focal: {focal:.1f} px  Ancho cara: {w} px")
        else:
            print("No se detecto ninguna cara al presionar C")

cap.release()
cv2.destroyAllWindows()
```

Líneas clave

- `cv2.CascadeClassifier(... "haarcascade_frontalface_default.xml")`: carga el modelo de rostros **frontales** que viene con OpenCV.
- `detectMultiScale(gris, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))` devuelve una lista de rectángulos (x, y, w, h) [6]:

Parámetro	Significado [6]
`scaleFactor`	Cuánto se reduce la imagen en cada escala (1.1 = 10 %). Así se buscan rostros de distintos tamaños.
`minNeighbors`	Cuántas detecciones vecinas debe tener un candidato para aceptarlo. Más alto = menos falsos positivos.
`minSize`	Tamaño mínimo del objeto; lo más pequeño se ignora.

- La distancia usa `ANCHO_CARA_CM = 14.0`, un ancho de cara supuesto para todas las personas. El texto se pinta verde si estás a menos de 50 cm y naranja si estás más lejos.

Juega con el código

E1 **minNeighbors**
Prueba `minNeighbors=1` y `minNeighbors=12`. ¿Con cuál aparecen "rostros" falsos y con cuál se pierden rostros reales?

E2 **scaleFactor**
Prueba `scaleFactor=1.05` y `1.5`. ¿Cambia la fluidez del video? ¿Detecta igual a distintas distancias?

E3 **De perfil**
Gira la cabeza poco a poco. ¿A partir de qué ángulo se pierde la detección? ¿Por qué, si el modelo se llama *frontalface*?

E4 **Otra persona**
Calibra con tu cara a 30 cm y luego pide a un compañero que se ponga a 30 cm. ¿Cuánto se equivoca y de dónde viene el error?

10. Del píxel a la acción del robot

Todos los scripts entregan resultados en **píxeles**: un centroide (cx, cy), un ancho w, una distancia estimada. Un robot, en cambio, se mueve en **milímetros** dentro de su propio sistema de coordenadas. El ciclo de la lámina "La visión en la robótica industrial" se completa así:

Etapa	Qué hace	Dónde lo viste
Sense	Capturar la imagen	`cap.read()` en todos los scripts
Think	Filtros → bordes → segmentación → centroide	`filtros.py`, `deteccion_bordes.py`, `deteccion_color.py`
Act	Convertir píxeles a mm y mover el robot	Requiere relacionar la cámara con el robot (calibración cámara-robot)

★ Ejemplo sencillo de conversión

Si la cámara mira la mesa desde arriba y su imagen de 640 × 480 px cubre un área de 320 × 240 mm, cada píxel equivale a 320 / 640 = **0,5 mm**. Un centroide en cx = 400 px está a (400 − 320) · 0,5 = **40 mm** a la derecha del centro de la imagen.

⚠ Limitaciones en la industria

Como se vio en clase, la visión es muy sensible a la luz, a las oclusiones (piezas tapadas) y a los reflejos metálicos; por eso las celdas industriales usan iluminación controlada y fondos uniformes. Las técnicas de la lámina "Detección de patrones" (transformada de Hough, SIFT) buscan identificar objetos aunque estén parcialmente tapados, rotados o a otra escala; no tienen script en esta guía.

11. Referencias

[1] OpenCV. *Changing Colorspaces* (rangos HSV y cómo hallar el valor HSV de un color). docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html

[2] OpenCV. *Canny Edge Detection*. docs.opencv.org/4.x/da/d22/tutorial_py_canny.html

[3] OpenCV. *Smoothing Images* (convolución y filtro gaussiano). docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html

[4] OpenCV. *Morphological Transformations* (erosión, dilatación, apertura y cierre). docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html

[5] OpenCV. *Contour Features* (momentos, centroide, área, boundingRect, minEnclosingCircle). docs.opencv.org/4.x/dd/d49/tutorial_py_contour_features.html

[6] OpenCV. *Cascade Classification* (detectMultiScale), docs.opencv.org/2.4.13.7/modules/objdetect/doc/cascade_classification.html, y *Cascade Classifier*, docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html. Viola, P. & Jones, M. (2001). Rapid Object Detection using a Boosted Cascade of Simple Features. IEEE CVPR.

[7] OpenCV. *Camera Calibration* (distancia focal y centro óptico). docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html

[8] Rosebrock, A. (2015). *Find distance from camera to object/marker using Python and OpenCV*. PyImageSearch.

[9] Matarić, M. J. (2007). *The Robotics Primer*. MIT Press. Cap. 9, sección 9.3 "Visual Sensing".

[10] OpenCV. *Histograms — Find, Plot, Analyze*. docs.opencv.org/4.x/d1/db7/tutorial_py_histogram_begins.html

[11] OpenCV. *Basic Operations on Images* (acceso a píxeles, shape, dtype, orden BGR). docs.opencv.org/4.x/d3/df2/tutorial_py_basic_ops.html